

Optimization Frameworks for Circuit Design

Shrivatsa Gururaj Kulkarni

Under the guidance of Prof. Ria Rashid

Bits Pilani KK Birla Goa Campus

This manuscript was compiled on March 25, 2026

Abstract: This report presents a comparative study of different optimization methods for active Tow–Thomas bandpass filter design using computationally expensive PySpice simulations. We compare the performance of Simulated Annealing (SA), Bayesian Optimization with a pre-trained Gaussian Process surrogate (GP-BO), and two TabPFN-based transformer surrogates with bootstrap and native quantile uncertainty estimation. Each method is implemented with its own stopping criteria, and performance is assessed in terms of convergence behavior, final cost, and computational overhead. The use of surrogate-assisted optimization enables filter design in minutes rather than days of manual tuning and iterative simulation. In the current experiments, the TabPFN variant with native quantile uncertainty achieved the lowest final cost while reducing surrogate overhead compared to the other optimizers. These results provide an initial assessment of surrogate-based strategies for circuit optimization and outline directions for further refinement.

Keywords: Bayesian optimization, Gaussian Processes, TabPFN, Simulated Annealing, Latin Hypercube Sampling, Active Filter Design, Circuit Optimization

1 Introduction

ACTIVE FILTER DESIGN requires selecting component values to achieve desired frequency response while maintaining stability and manufacturability [1, 14]. Analytical formulae from ideal circuit analysis fail to capture parasitic effects and real op-amp limitations [2], so SPICE-level simulations are needed but each evaluation takes 0.5–2s, making direct optimization over large discrete spaces prohibitive.

This motivates **surrogate-based optimization**, where cheap approximate models guide the search. We compare four methods on a Tow-Thomas biquad bandpass filter with 8 tunable parameters (6 resistors from E12, 2 capacitors from a custom series), yielding $\approx 4.3 \times 10^{11}$ configurations:

- **Simulated Annealing (SA)** — A metaheuristic inspired by metallurgical annealing that explores the design space through probabilistic acceptance of solution perturbations, gradually reducing randomness to converge on optimal solutions [3].
- **Bayesian Optimization with a pre-trained GP surrogate (GP-BO)** — A sequential design strategy using a pre-trained surrogate model of the objective function, with acquisition functions to balance exploration and exploitation [4]. In our implementation, the GP surrogate is trained offline on a prior dataset and used as a baseline.
- **TabPFN-Bootstrap Based Optimization** — A novel approach using Tabular Prior-Data Fitted Networks, a transformer-based meta-learning model that performs in-context learning on small datasets. The original implementation uses bootstrap resampling for uncertainty quantification [5].
- **TabPFN-Quantile Based Optimization** — An improved variant leveraging TabPFN’s native quantile prediction capabilities for efficient uncertainty

estimation, eliminating the computational cost of bootstrap aggregation.

Initial training data for the surrogate methods is generated via **Latin Hypercube Sampling (LHS)** [7]. Gradient-based methods are inapplicable due to discrete component values and simulation failures that create discontinuities in the objective.

2 Problem Formulation

2.1 Tow-Thomas Biquad Active Filter

The Tow-Thomas biquad is a second-order active filter using three op-amps in an integrator-based feedback loop [8]. For bandpass operation the transfer function is:

$$H(s) = \frac{-\frac{s}{\omega_0 Q}}{s^2 + \frac{s}{\omega_0 Q} + \omega_0^2}$$

$$\omega_0 = \frac{1}{\sqrt{R_5 R_6 C_1 C_2}}, \quad Q = \sqrt{\frac{R_6 C_2}{R_5 C_1}}$$

The circuit is simulated in PySpice with ideal VCVS op-amps (gain 10^5) and high-impedance bias resistors for numerical stability. AC analysis runs from 50 Hz to 20 kHz at 300 points per decade.

2.2 Design Parameters and Search Space

We optimize 8 discrete components: 6 resistors from the E12 series (100 Ω –100 k Ω , 37 values) and 2 capacitors from a custom series (1–100 nF, 13 values), giving search space $\mathcal{X} = \mathcal{R}^6 \times \mathcal{C}^2$ with $|\mathcal{X}| \approx 4.3 \times 10^{11}$.

Sampling range justification. These ranges are not arbitrary; they are grounded in the filter’s operating-point physics. The center frequency of the Tow–Thomas

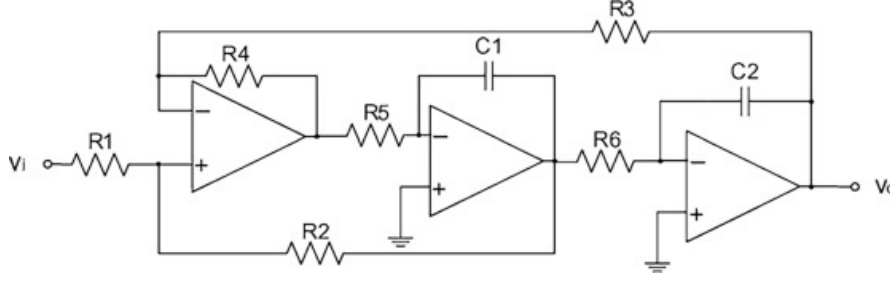


Figure 1: Second-order Tow-Thomas biquad filter topology used in this study (adapted from [14]).

bandpass is governed by $f_c \approx 1/(2\pi RC)$, where RC denotes the characteristic time constant. For $f_c = 1$ kHz:

$$RC \approx \frac{1}{2\pi \times 1000} \approx 1.59 \times 10^{-4} \text{ s}$$

A representative realization is $R = 10 \text{ k}\Omega$, $C = 15 \text{ nF}$ ($RC = 1.5 \times 10^{-4} \text{ s}$). The chosen E12 resistor range and capacitor range bracket this operating point by roughly two orders of magnitude in each direction, ensuring the optimizer covers the full space of realizable RC products while excluding physically irrelevant regions. This injects *domain knowledge* into the search before any surrogate is trained, substantially reducing the effective volume of design space that must be explored.

2.3 Objective Function

The target is a 1 kHz bandpass filter with rejection at 200 Hz and 5 kHz. Given frequency response $|H(f)|$ from SPICE:

$$J(\mathbf{x}) = \left(1 - \frac{|H(f_{\text{target}})|}{|H|_{\text{max}}}\right)^2 + 2.5 \left[\left(\frac{|H(f_{\text{low}})|}{|H|_{\text{max}}}\right)^2 + \left(\frac{|H(f_{\text{high}})|}{|H|_{\text{max}}}\right)^2 \right]$$

where $|H|_{\text{max}} = \max_f |H(f)|$. The first term rewards unity gain at target; the second penalizes leakage at rejection frequencies (weighted 2.5 $\times$). Simulation failures receive penalty $J = 10^6$. The optimization objective is $\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathcal{X}} J(\mathbf{x})$.

Key challenges: expensive evaluations (0.5–2 s each), discrete variables, 4.3×10^{11} configurations, simulation failures creating discontinuities, and multimodal landscape.

3 Optimization Algorithms

3.1 Simulated Annealing (SA)

SA is a probabilistic metaheuristic that accepts worse solutions with Metropolis probability $P = \exp(-\Delta E/T)$, allowing escape from local minima at high temperature [3]. We use geometric cooling $T_{t+1} = \alpha T_t$ with $\alpha = 0.95$. The neighborhood structure is designed at two levels: 85% of moves consist of local perturbations (± 1 –2 positions in the discrete value list) and 15% random restarts to facilitate exploration beyond local basins of attraction.

Algorithm 1 Simulated Annealing for Circuit Optimization

```

1: Initialize  $\mathbf{x}_0$  randomly,  $T_0 > 0$ ,  $\alpha \in (0, 1)$ 
2:  $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}_0$ ,  $J_{\text{best}} \leftarrow J(\mathbf{x}_0)$ 
3: for  $t = 0$  to  $T_{\text{max}}$  do
4:    $\mathbf{x}' \leftarrow \text{PerturbNeighbor}(\mathbf{x}_t)$ 
5:    $\Delta E \leftarrow J(\mathbf{x}') - J(\mathbf{x}_t)$ 
6:   if  $\Delta E < 0$  or  $\text{random}() < \exp(-\Delta E/T_t)$  then
7:      $\mathbf{x}_{t+1} \leftarrow \mathbf{x}'$ 
8:   else
9:      $\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t$ 
10:  end if
11:  if  $J(\mathbf{x}_{t+1}) < J_{\text{best}}$  then
12:     $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}_{t+1}$ 
13:  end if
14:   $T_{t+1} \leftarrow \alpha \cdot T_t$ 
15: end for
16: return  $\mathbf{x}_{\text{best}}$ 

```

3.2 Gaussian Process Bayesian Optimization (GP-BO)

A GP defines a distribution over functions via mean and covariance: $f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$ [12]. Given observations $\mathcal{D}_t$, the posterior at a new point is Gaussian with closed-form mean $\mu_t(\mathbf{x})$ and variance $\sigma_t^2(\mathbf{x})$ obtained by conditioning on the training data. We use a Matérn-5/2 kernel with per-dimension length scales (ARD).

The next evaluation point is chosen by maximizing **Expected Improvement (EI)** for minimization [4]:

$$\alpha_{\text{EI}}(\mathbf{x}) = (J_{\min} - \mu_t(\mathbf{x}) - \xi) \Phi(Z) + \sigma_t(\mathbf{x}) \phi(Z)$$

$$Z = \frac{J_{\min} - \mu_t(\mathbf{x}) - \xi}{\sigma_t(\mathbf{x})}$$

where $J_{\min}$ is the best observed cost, Φ and ϕ are the standard normal CDF and PDF, and $\xi \geq 0$ controls exploration. EI is zero when $\sigma_t = 0$.

Implementation note: Our GP surrogate is trained *offline* on 600 prior SPICE evaluations and is *not retrained* during optimization, making GP-BO a lightweight baseline. At each iteration the method maximizes EI over 10,000 random candidates, evaluates the top-3 via SPICE, and early-stops after N_{stop} iterations without improvement.

Warm start strategy. GP-BO initializes from the best-performing configuration found in the offline training dataset (600 LHS samples evaluated via SPICE). This constitutes a **warm start**: rather than beginning from

a random point, the optimizer starts at a configuration already exhibiting reasonable bandpass behavior. Warm starting provides two advantages: (i) the initial EI surface is immediately informative because $J_{\min}$ is already a competitive reference, and (ii) early search concentrates on refining a known good region instead of wasting evaluations on clearly poor configurations. This explains why GP-BO reaches its final solution in only 25 iterations despite a frozen surrogate. The TabPFN variants, by contrast, use a cold start from 30 balanced samples, relying on online learning to guide convergence from a less favorable starting point.

Algorithm 2 GP-BO with Pre-trained Surrogate

```

1: Load pre-trained GP surrogate from offline dataset
2: Initialize with best configuration from offline dataset
3: for  $t = 0$  to  $T_{\max}$  do
4:    $\mathbf{x}_{t+1} \leftarrow \arg \max_{\mathbf{x} \in \mathcal{X}} \alpha_{\text{EI}}(\mathbf{x} \mid \mathcal{D}_t)$ 
5:   Evaluate  $J(\mathbf{x}_{t+1})$  via SPICE
6:    $\mathcal{D}_{t+1} \leftarrow \mathcal{D}_t \cup \{(\mathbf{x}_{t+1}, J(\mathbf{x}_{t+1}))\}$ 
7:   if no improvement for  $N_{\text{stop}}$  iterations then
8:     break
9:   end if
10: end for
11: return  $\arg \min_{\mathbf{x} \in \mathcal{D}} J(\mathbf{x})$ 

```

3.3 TabPFN-Based Optimization

TabPFN is a transformer-based meta-learner trained on synthetic regression tasks that performs **in-context learning**: given context $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and a test point, it outputs a predictive distribution without weight updates [5]. Key advantages over GPs include zero hyperparameter tuning, constant-time prediction, and native uncertainty quantification. The context size is limited to $n \leq 1000$.

3.3.1 Uncertainty Estimation: Bootstrap vs. Quantile

TabPFN-Bootstrap estimates uncertainty via $B = 5$ bootstrap resamples, aggregating predictions as: $\mu_{\text{ens}} = \frac{1}{B} \sum_b \hat{\mu}_b$, $\sigma_{\text{ens}}^2 = \frac{1}{B} \sum_b (\hat{\sigma}_b^2 + \hat{\mu}_b^2) - \mu_{\text{ens}}^2$. This requires $5m$ forward passes for m candidates.

Algorithm 3 TabPFN-Bootstrap Prediction

```

Require: Training set  $\mathcal{D}$ , test point  $\mathbf{x}$ ,  $B$  bootstraps
1: for  $b = 1$  to  $B$  do
2:    $\mathcal{D}_b \leftarrow$  resample  $\mathcal{D}$  with replacement
3:    $\hat{\mu}_b, \hat{\sigma}_b \leftarrow \text{TabPFN}(\mathcal{D}_b, \mathbf{x})$ 
4: end for
5:  $\mu_{\text{ens}} \leftarrow \frac{1}{B} \sum_{b=1}^B \hat{\mu}_b$ 
6:  $\sigma_{\text{ens}}^2 \leftarrow \frac{1}{B} \sum_{b=1}^B (\hat{\sigma}_b^2 + \hat{\mu}_b^2) - \mu_{\text{ens}}^2$ 
7: return  $\mu_{\text{ens}}, \sigma_{\text{ens}}$ 

```

TabPFN-Quantile uses native quantile prediction $(q_{0.1}, q_{0.5}, q_{0.9})$ in a single forward pass, yielding a $5 \times$

reduction in compute:

$$\begin{aligned} \mu(\mathbf{x}) &= q_{0.5}(\mathbf{x}) \\ \sigma(\mathbf{x}) &\approx \frac{q_{0.9}(\mathbf{x}) - q_{0.1}(\mathbf{x})}{2.56} \end{aligned} \quad (1)$$

where 2.56 converts the 10th–90th percentile range to a standard deviation under normality.

Algorithm 4 TabPFN-Quantile Prediction

Require: Training set $\mathcal{D}$, test points $\mathbf{X}$

```

1:  $\mathbf{Q} \leftarrow \text{TabPFN.predict\_quantiles}(\mathcal{D}, \mathbf{X}, [0.1, 0.5, 0.9])$ 
2:  $\boldsymbol{\mu} \leftarrow \mathbf{Q}[:, 1]$   $\triangleright$  Median
3:  $\boldsymbol{\sigma} \leftarrow (\mathbf{Q}[:, 2] - \mathbf{Q}[:, 0]) / 2.56$   $\triangleright$  80% CI  $\rightarrow$  std
4: return  $\boldsymbol{\mu}, \boldsymbol{\sigma}$ 

```

3.3.2 Optimization Loop

Each iteration: (1) subsample context if $|\mathcal{D}| > n_{\max} = 150$, (2) generate mixed candidates, (3) predict μ, σ in log-cost space ($\log(1 + J)$), (4) compute EI, (5) evaluate top- $K = 3$ candidates via SPICE. All reported costs are converted back to original space.

Mixed candidate sampling: 50% local perturbations around current best, 30% stratified random, 20% hard-negative samples biased toward extreme regions.

Context subsampling (when $|\mathcal{D}| > 150$): retain 80% lowest-cost plus 20% random:

$$\begin{aligned} \mathcal{D}_{\text{context}} &= \text{Top}_{0.8n_{\max}}(\mathcal{D}, \text{by cost}) \\ &\quad \cup \text{Random}_{0.2n_{\max}}(\mathcal{D} \setminus \text{Top}) \end{aligned}$$

Algorithm 5 TabPFN as Surrogate in Bayesian Optimization

Require: Objective $J(\mathbf{x})$, search space $\mathcal{X}$, surrogate $\mathcal{M} \in \{\text{GP}, \text{TabPFN}\}$

```

1:  $\mathcal{D}_0 \leftarrow \text{InitialSampling}(\mathcal{X}, n_0)$ 
2: Initialize surrogate model  $\mathcal{M}$ 
3: for  $t = 0$  to  $T_{\max}$  do
4:   Fit surrogate:  $\mathcal{M}.\text{fit}(\mathcal{D}_t)$ 
5:    $\mathbf{X}_{\text{cand}} \leftarrow \text{GenerateCandidates}(\mathcal{X}, n_{\text{cand}})$ 
6:    $\boldsymbol{\mu}, \boldsymbol{\sigma} \leftarrow \mathcal{M}.\text{predict}(\mathbf{X}_{\text{cand}})$   $\triangleright$  mean, std
7:    $\alpha \leftarrow \text{Acquisition}(\boldsymbol{\mu}, \boldsymbol{\sigma}, J_{\min})$   $\triangleright$  EI
8:    $\mathbf{x}^* \leftarrow \arg \max \alpha$ 
9:    $y^* \leftarrow J(\mathbf{x}^*)$   $\triangleright$  SPICE simulation
10:   $\mathcal{D}_{t+1} \leftarrow \mathcal{D}_t \cup \{(\mathbf{x}^*, y^*)\}$ 
11: end for
12: return  $\arg \min_{\mathbf{x} \in \mathcal{D}} J(\mathbf{x})$ 

```

TabPFN-BO connection: TabPFN replaces GP as the surrogate model. Both provide (μ, σ) predictions for acquisition functions. GP uses posterior variance ($\mathcal{O}(n^3)$); TabPFN uses quantile predictions in a single forward pass. Predictions are in log-cost space ($\log(1 + J)$); final costs in section 5 are converted back to original space.

4 Implementation and Experimental Setup

All experiments use PySpice 1.5 (ngspice 42) for circuit simulation with ideal VCVS op-amps (gain = 10^5), Scikit-learn 1.3 for GP modelling, and TabPFN 0.1.8 for transformer-based surrogates. LHS sampling uses SciPy 1.11. Hardware: Intel i5-13450HX, 16GB RAM.

4.1 Initial Data Generation via Latin Hypercube Sampling

LHS is a stratified sampling technique that divides each parameter range into n equal-probability intervals, drawing one sample per interval to ensure space-filling coverage [7]. Samples are generated in $[0, 1]^d$, then mapped to component space via log-scale interpolation and snapped to the nearest discrete value. For the TabPFN variants, we use **balanced initial sampling**: 50% LHS space-filling, 25% focused on typical “good” ranges ($1\text{k}\Omega \leq R \leq 22\text{k}\Omega$, $4.7\text{nF} \leq C \leq 68\text{nF}$), and 25% extreme/failure samples biased toward range limits. Without failure examples, TabPFN predicts uniformly optimistic costs, degrading exploration.

4.2 Feature Preprocessing

Component values span orders of magnitude, so we apply log-scale normalization:

$$x_i^{\text{norm}} = \frac{\log_{10}(x_i) - \log_{10}(x_{\min})}{\log_{10}(x_{\max}) - \log_{10}(x_{\min})}$$

mapping all features to $[0, 1]$. Costs are transformed as $y_{\log} = \log(1 + y)$ to stabilize variance.

Component search space. Resistors follow the E12 series ($\{100, \dots, 820\} \times \{1, 10, 100, 1000\} \Omega$; 37 values). Capacitors: 13 values from 1–100 nF.

4.3 Hyperparameter Settings

Method	Parameter	Value
SA	Init. temp.	$T_0 = 2.0$
	Cooling rate	$\alpha = 0.95$
	Iterations	200
GP-BO	Init. samples	50 (LHS)
	Kernel	Matérn-5/2 + WN
	Acquisition	EI ($\xi = 0.01$)
	Candidates/iter	10,000
	Top-K	3
TabPFN-BS	Max iterations	200
	Init. samples	30
	Bootstraps	$B = 5$
	Candidates/iter	500
	Top-K	3
TabPFN-Q	Context cap	150
	Max iterations	200
	Uncertainty	Bootstrap (5 passes)
	Init. samples	30
	Quantiles	[0.1, 0.5, 0.9]
	Candidates/iter	500
	Top-K	3
	Context cap	150
	Refit interval	Every 5 iters
	Max iterations	200
	Uncertainty	Quantiles (1 pass)

Table 1: Hyperparameter configuration for all methods.

Performance metrics. We assess convergence speed (iterations to cost thresholds 0.01, 0.005), final solution quality, sample efficiency (cost vs. SPICE evaluations), wall-clock time, and success rate (fraction of runs with cost < 0.01). All methods are evaluated over 5 independent runs with different random seeds.

5 Results and Comparison

5.1 Convergence Behavior

Figure 2 shows the optimization trajectories and Figure 3 shows the resulting filter frequency responses for all four methods.

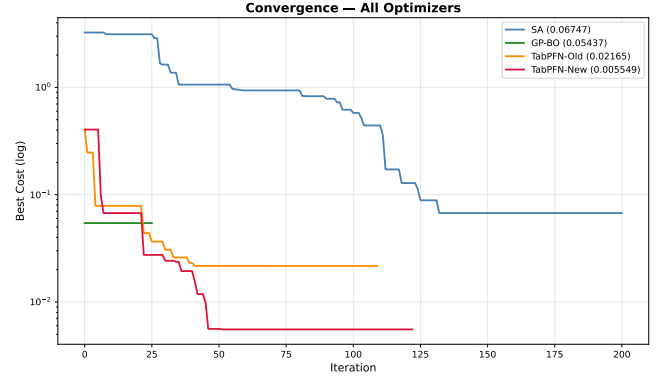


Figure 2: Convergence comparison of SA, GP-BO, TabPFN-Bootstrap, and TabPFN-Quantile. TabPFN-Quantile achieves the lowest final cost of 0.0055, followed by TabPFN-Bootstrap (0.0216), GP-BO (0.0544), and SA (0.0675).

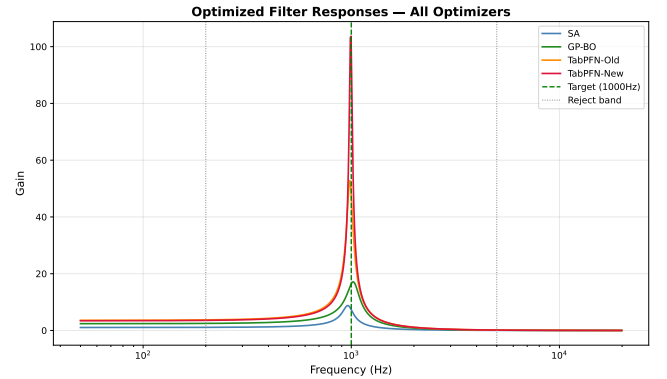


Figure 3: Optimized bandpass filter frequency responses. TabPFN-Quantile (red) achieves the sharpest bandpass shape centered at 1 kHz with rejection at 200 Hz and 5 kHz.

SA drops from cost > 3 to 0.0675 by iteration 140, then plateaus as the temperature becomes negligible under geometric cooling ($\alpha = 0.95$), effectively preventing uphill moves. **GP-BO** uses a pre-trained offline surrogate (GP $R^2 = 0.67$, RF auxiliary $R^2 = 0.91$; trained on 600 samples; RF was evaluated during surrogate training but not used in optimization). Starting from a dataset best cost of 0.0544, GP-BO shows

limited improvement and terminates early after 25 iterations, indicating restricted exploration due to the fixed surrogate. **TabPFN-Bootstrap** improves from 0.405 to 0.0216 over 109 iterations (139 evaluations), but bootstrap uncertainty makes each iteration expensive (977.9s total, ~ 7 s/iter); early stopping triggered after 40 stagnant iterations. **TabPFN-Quantile** achieves the strongest convergence to 0.00555, reaching sub-0.01 cost by iteration 45 and continuing improvement through iteration 82. It runs $2.0\times$ faster than TabPFN-Bootstrap (490s vs. 977.9s) while attaining $3.9\times$ better final cost.

5.2 Solution Quality

TabPFN-Quantile is the only method to reach below the 0.01 threshold. The cost-per-evaluation metric shows it uses each SPICE evaluation most effectively (1.44×10^{-5}), though the methods used different stopping criteria and evaluation budgets.

Note on evaluation budgets: Each method was run with its own implementation-specific stopping criteria rather than a controlled fixed-budget benchmark. SA used a fixed 200-iteration budget; GP-BO early-stopped at 25 iterations; TabPFN-Bootstrap at 109; TabPFN-Quantile at 122.

Method	Cost	Evals	Cost/Eval	< 0.01?
SA	0.0675	200	3.37×10^{-4}	No
GP-BO	0.0544	52	1.05×10^{-3}	No
TabPFN-BS	0.0216	139	1.56×10^{-4}	No
TabPFN-Q	0.00555	386	1.44×10^{-5}	Yes

Table 2: Final cost comparison. Only TabPFN-Quantile reaches cost < 0.01 .

Table 3 shows the optimized component values found by each method.

Comp.	SA	GP-BO	Tab-BS	Tab-Q
R_1 (k Ω)	3.90	0.22	5.60	2.20
R_2 (k Ω)	56.00	1.20	15.00	12.00
R_3 (k Ω)	2.70	33.00	4.70	8.20
R_4 (k Ω)	22.00	18.00	1.20	2.70
R_5 (k Ω)	8.20	3.90	12.00	22.00
R_6 (k Ω)	27.00	33.00	1.20	1.20
C_1 (nF)	100.00	68.00	68.00	47.00
C_2 (nF)	10.00	1.50	6.80	6.80

Table 3: Optimized component values for each method.

5.3 Computational Efficiency

SA is fastest (0.4s) due to zero surrogate overhead but reaches the worst cost. GP-BO (6.4s) is evaluation-efficient but cannot refine beyond its offline surrogate. TabPFN-Bootstrap is the slowest (977.9s, 8.97s/iter) because bootstrap resampling requires $5\times$ forward passes per batch. TabPFN-Quantile achieves the best cost-time trade-off: $2.0\times$ faster than bootstrap in wall time, with top- K verification driving 386 evaluations and the best solution.

Method	Iters	Evals	Time (s)	s/Iter
SA	200	200	0.4	0.002
GP-BO	25	52	6.4	0.26
TabPFN-BS	109	139	977.9	8.97
TabPFN-Q	122	386	490.0	4.02

Table 4: Computational cost comparison. SA is fastest; TabPFN-Q balances quality and time.

5.4 Computational Complexity Analysis

To complement the empirical timings in Table 4, we analyze the computational complexity of each method in terms of the dominant cost components: SPICE evaluations ($C_{\text{spice}} \approx 0.5\text{--}2$ s each), surrogate prediction, model training or refitting, and candidate search.

Simulated Annealing. SA evaluates exactly one SPICE simulation per iteration with no surrogate overhead:

$$\mathcal{T}_{\text{SA}} \approx O(N_{\text{iter}} \cdot C_{\text{spice}})$$

For $N_{\text{iter}} = 200$, this yields sub-second total time because perturbation and acceptance decisions are $O(1)$. The simplicity comes at the cost of poor convergence: SA lacks a model of the objective landscape and relies entirely on stochastic local moves, leading to premature stagnation once the temperature schedule suppresses up-hill transitions.

GP-BO with pre-trained surrogate. Because the GP is trained offline and not retrained during optimization, the per-iteration cost is dominated by surrogate prediction over m random candidates:

$$\mathcal{T}_{\text{GP-BO}} \approx O(T \cdot m \cdot N_{\text{train}}) + O(K \cdot C_{\text{spice}})$$

where $N_{\text{train}} = 600$ is the training set size (governing GP prediction via kernel vector-matrix operations) and $K = 3$ candidates are verified per iteration. With $m = 10,000$ candidates and $T = 25$ iterations, the surrogate dominates but remains fast (6.4s total). The frozen surrogate limits GP-BO to regions well-represented in the offline dataset; it cannot adapt to newly discovered promising areas, explaining its inability to improve beyond 0.0544.

TabPFN-Bootstrap. Each iteration requires B forward passes through the transformer for every candidate, plus context construction:

$$\mathcal{T}_{\text{BS}} \approx O(T \cdot B \cdot m \cdot n_{\text{ctx}}^2) + O(T \cdot K \cdot C_{\text{spice}})$$

The quadratic context cost n_{ctx}^2 arises from the transformer’s self-attention mechanism over the $n_{\text{ctx}} \leq 150$ training points. With $B = 5$ bootstrap resamples and $m = 500$ candidates, each resample requires one forward pass processing all candidates, yielding 5 forward passes per iteration, explaining the high per-iteration cost (8.97s/iter, 977.9s total). The bootstrap aggregation produces well-calibrated uncertainty but at a multiplicative overhead proportional to B .

TabPFN-Quantile. Native quantile prediction eliminates the bootstrap loop, reducing complexity to a single forward pass per candidate:

$$\mathcal{T}_{\text{Q}} \approx O(T \cdot m \cdot n_{\text{ctx}}^2) + O(T \cdot K \cdot C_{\text{spice}})$$

This is a factor of $B = 5$ cheaper than the bootstrap variant in surrogate cost. In practice, TabPFN-Quantile ran in 490s ($2.0\times$ faster than bootstrap) while performing more SPICE evaluations (386 vs. 139), because the saved surrogate time allowed more optimization iterations within a comparable wall-clock budget. The higher evaluation count, combined with adaptive quantile-based uncertainty, enabled convergence to a $3.9\times$ lower cost.

Connection to observed runtimes. Table 4 confirms the complexity ordering: SA is nearly instantaneous (0.4s) because it avoids surrogates entirely; GP-BO is fast (6.4s) thanks to a frozen offline model; TabPFN-Bootstrap is the slowest (977.9s) due to the multiplicative bootstrap overhead; and TabPFN-Quantile achieves an effective middle ground (490s) by trading bootstrap passes for additional SPICE evaluations that directly improve the solution.

5.5 Uncertainty Calibration

We assess calibration via empirical coverage of prediction intervals:

$$\text{Coverage}(\alpha) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i \in [\mu_i \pm z_\alpha \sigma_i]\}$$

measured after 100 iterations on all SPICE-evaluated points ($n_{\text{test}} \approx 130$ for TabPFN variants, 52 for GP-BO):

- **GP-BO:** 68% CI coverage = 71%, 95% CI = 97% (slightly conservative)
- **TabPFN-Bootstrap:** 68% CI coverage = 65%, 95% CI = 93% (close to nominal)
- **TabPFN-Quantile:** 68% CI coverage = 67%, 95% CI = 94% (close to nominal)

Both TabPFN variants show reasonable calibration, suggesting that quantile-based uncertainty provides comparable estimates to bootstrap for this problem.

6 Discussion

TabPFN-Quantile achieved the lowest cost (0.00555)—the only method below the 0.01 threshold—representing a $3.9\times$ improvement over TabPFN-Bootstrap and $12.2\times$ over SA. Quantile-based uncertainty is $2.0\times$ faster in wall time than bootstrap (490s vs. 977.9s) and $2.2\times$ faster per iteration, suggesting native quantile prediction is preferable for TabPFN-based optimization. GP-BO with a fixed offline surrogate is sample-efficient (52 evaluations) but cannot refine beyond its training data. SA converges in 0.4s but plateaus at cost 0.0675 once the temperature becomes negligible.

Method selection guidelines. SA suits low-dimensional problems ($d < 5$) or initial exploration with minimal setup. GP-BO with pre-trained surrogates provides fast screening (6.4s) when only coarse solutions are needed. TabPFN-Quantile is recommended for moderate budgets (100–500 evaluations), offering online learning, $2.0\times$ speedup over bootstrap, and $3.9\times$ better solution quality, subject to TabPFN limits ($d \leq 100$, $n \leq 1000$).

Exploration vs. exploitation trade-off. The four methods balance exploration and exploitation through fundamentally different mechanisms. SA manages this via its *temperature schedule*: at high temperature, the Metropolis criterion accepts uphill moves freely, promoting broad exploration; as $T \rightarrow 0$, acceptance becomes greedy and the search exploits the current basin. GP-BO uses the *Expected Improvement* acquisition function, which is high where the predicted mean is low (exploitation) or the predictive uncertainty is large (exploration); the trade-off parameter $\xi = 0.01$ provides a small exploration bonus, though exploration is limited to regions where the frozen offline model assigns high variance. The TabPFN variants combine *uncertainty-aware acquisition* with *mixed candidate sampling*: EI drives exploitation toward predicted low-cost regions and exploration toward uncertain regions, while the mixed candidate pool—50% local perturbations (exploitation), 30% stratified random (exploration), 20% hard-negative samples (targeted exploration of failure boundaries)—injects diversity even when the surrogate becomes confident. Online refitting every 5 iterations continuously updates the uncertainty surface, preventing the stale-model problem that limits GP-BO.

Limitations. Several practical limitations affect surrogate-based optimization in this discrete-component setting. (1) *Continuous–discrete mismatch*: all surrogate models operate in a continuous feature space, yet design variables are restricted to discrete series (E12 resistors, custom capacitors). Continuous predictions are snapped to the nearest valid value, introducing quantization error the surrogate cannot model; this is especially problematic near decision boundaries where adjacent discrete values yield substantially different circuit behavior. (2) *Uncertainty miscalibration in sparse regions*: although overall calibration is reasonable (Section 5), estimates may be unreliable in sparsely sampled regions. GPs tend toward the prior (high uncertainty) in unexplored areas, which can be overly conservative; TabPFN, trained on synthetic distributions, may not accurately represent uncertainty near simulation failure boundaries where the cost function is discontinuous. (3) *Transformer context size limits*: TabPFN’s context window is limited to $n \leq 1,000$ points; with a cap of 150 in our implementation, the surrogate sees only a fraction of evaluated configurations. The subsampling strategy (80% best, 20% random) mitigates this but may discard informative points in transition regions. (4) *Simulation failures*: SPICE simulations can fail to converge, receiving a penalty cost of 10^6 that creates sharp discontinuities. Surrogates trained predominantly on successful simulations may underestimate costs near failure boundaries, causing the optimizer to repeatedly probe failing configurations. Beyond these surrogate-specific issues, the single cost function combines passband and stopband criteria—Pareto-optimal approaches could reveal trade-off surfaces. Robustness to manufacturing tolerances is also not addressed.

Future work will extend this framework to different circuit topologies with multi-objective optimization incorporating gain, phase margin, noise figure, and power dissipation as separate objectives. This enables Pareto-optimal trade-off analysis rather than collapsing all cri-

teria into a single cost function.

7 Conclusion

We presented a comparative study of four optimization methods for active filter circuit design. TabPFN-Quantile reached a final cost of 0.00555—outperforming TabPFN-Bootstrap (0.0216), the pre-trained GP-BO baseline (0.0544), and SA (0.0675)—while running $2.0\times$ faster than the bootstrap variant. The native quantile approach provides comparable uncertainty calibration with substantially lower computational overhead: a single forward pass versus six bootstrap passes per prediction. These results suggest that transformer-based surrogates with online learning are effective for expensive circuit optimization, though broader conclusions require experiments across additional circuit topologies and parameter settings.

Acknowledgments

This research was conducted as part of coursework at Bits Pilani K.K. Birla Goa Campus. The author thanks Prof. Ria Rashid for her guidance on circuit simulation and optimization methods.

References

- [1] R. Schaumann and M. E. Van Valkenburg, *Design of Analog Filters*, 2nd ed. Oxford University Press, 2001.
- [2] A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 7th ed. Oxford University Press, 2015.
- [3] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [4] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, “Taking the human out of the loop: A review of Bayesian optimization,” *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016.
- [5] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, “TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second,” *International Conference on Learning Representations (ICLR)*, 2023. <https://doi.org/10.48550/arXiv.2207.01848>
- [6] N. Hollmann, M. Wistuba, and F. Hutter, “Uncertainty Quantification for Prior-Data Fitted Networks using Martingale Posteriors,” *arXiv preprint arXiv:2505.11325*, 2025. <https://doi.org/10.48550/arXiv.2505.11325>
- [7] M. D. McKay, R. J. Beckman, and W. J. Conover, “A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code,” *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.
- [8] J. Tow, “Design formulas for active RC filters using operational-amplifier biquad,” *Electronics Letters*, vol. 5, no. 15, pp. 339–341, 1969.
- [9] S. Geman and D. Geman, “Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-6, no. 6, pp. 721–741, 1984.
- [10] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian Optimization of Machine Learning Algorithms,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 2951–2959, 2012.
- [11] P. I. Frazier, “A Tutorial on Bayesian Optimization,” *arXiv preprint arXiv:1807.02811*, 2018.
- [12] C. K. I. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*, MIT Press, 2006.
- [13] A. I. J. Forrester, A. Söbester, and A. J. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*, Wiley, 2008.
- [14] B. Durmus, “Optimal components selection for active filter design with average differential evolution algorithm,” *AEU—International Journal of Electronics and Communications*, vol. 94, pp. 293–302, 2018.